

# The AWS Security Playbook

A complete, detailed guide to securing and assessing AWS environments — from IAM fundamentals to incident response

*From a single overly permissive IAM policy to a full account compromise, this playbook builds the cloud security intuition and detection fluency that securing AWS environments depends on.*

**Author:** Swastik Sagar  
Final-year Computer Science Undergraduate  
SOC Analyst Intern

August 16, 2026

# Preface

Cloud security incidents rarely start with a novel exploit — they start with a misconfiguration: an S3 bucket left public, an IAM role with far broader permissions than it needs, credentials committed to a public repository, a security group open to the entire internet. Understanding AWS security well means understanding the identity and access model deeply enough to spot these gaps before they're found by someone else, and knowing exactly what AWS's own logging and detection tools already give you for free.

This playbook is organized around that reality: IAM first, since almost everything else in AWS security is downstream of identity and permissions, then the services most frequently misconfigured (S3, EC2, VPC), then AWS's native logging and detection tooling (CloudTrail, GuardDuty), then how these pieces come together into real attack chains, and finally incident response specific to a cloud environment.

## Authorization required

Every technique in this book is intended strictly for authorized security testing and assessment of AWS accounts and resources you own or have explicit written authorization to test. Testing AWS resources you don't own or control is illegal in most jurisdictions and a violation of AWS's own Acceptable Use Policy.

## What this book covers

- IAM fundamentals: users, roles, policies, and how permissions are actually evaluated
- Common IAM privilege escalation paths
- S3 security and the misconfigurations that lead to public data exposure
- EC2 and compute security, including the instance metadata service
- VPC and network security fundamentals
- CloudTrail: what it logs and how to use it
- GuardDuty and native threat detection
- Secrets management done correctly
- Common cloud attack chains, end to end
- Security Hub and compliance tooling
- Practical detection recipes
- Incident response specific to AWS

*Swastik Sagar*

# Contents

|                                                            |          |
|------------------------------------------------------------|----------|
| <b>Preface</b>                                             | <b>i</b> |
| <b>1 IAM Fundamentals</b>                                  | <b>1</b> |
| 1.1 Why IAM is the foundation . . . . .                    | 1        |
| 1.2 Core IAM concepts . . . . .                            | 1        |
| 1.3 How permissions are actually evaluated . . . . .       | 1        |
| 1.4 Reading a policy document . . . . .                    | 2        |
| 1.5 The principle of least privilege . . . . .             | 2        |
| <b>2 IAM Privilege Escalation</b>                          | <b>3</b> |
| 2.1 Why IAM escalation matters . . . . .                   | 3        |
| 2.2 Common escalation paths . . . . .                      | 3        |
| 2.3 The fix . . . . .                                      | 3        |
| <b>3 S3 Security</b>                                       | <b>4</b> |
| 3.1 Why S3 misconfigurations are so common . . . . .       | 4        |
| 3.2 Layers of S3 access control . . . . .                  | 4        |
| 3.3 Finding exposed buckets during an assessment . . . . . | 4        |
| 3.4 The fix . . . . .                                      | 5        |
| <b>4 EC2 and Compute Security</b>                          | <b>6</b> |
| 4.1 The instance metadata service . . . . .                | 6        |
| 4.2 IMDSv1 vs. IMDSv2 . . . . .                            | 6        |
| 4.3 Security groups . . . . .                              | 6        |
| 4.4 The fix . . . . .                                      | 7        |
| <b>5 VPC and Network Security</b>                          | <b>8</b> |
| 5.1 Core VPC concepts . . . . .                            | 8        |
| 5.2 Public vs. private subnets . . . . .                   | 8        |
| 5.3 VPC Flow Logs . . . . .                                | 8        |

|           |                                                              |           |
|-----------|--------------------------------------------------------------|-----------|
| <b>6</b>  | <b>CloudTrail</b>                                            | <b>9</b>  |
| 6.1       | What CloudTrail records . . . . .                            | 9         |
| 6.2       | Management vs. data events . . . . .                         | 9         |
| 6.3       | Key fields for investigation . . . . .                       | 9         |
| 6.4       | Querying CloudTrail . . . . .                                | 10        |
| <b>7</b>  | <b>GuardDuty and Threat Detection</b>                        | <b>11</b> |
| 7.1       | What GuardDuty does . . . . .                                | 11        |
| 7.2       | Finding categories . . . . .                                 | 11        |
| 7.3       | Responding to a GuardDuty finding . . . . .                  | 11        |
| <b>8</b>  | <b>Secrets Management</b>                                    | <b>12</b> |
| 8.1       | Where secrets go wrong . . . . .                             | 12        |
| 8.2       | AWS Secrets Manager and Parameter Store . . . . .            | 12        |
| <b>9</b>  | <b>Common Cloud Attack Chains</b>                            | <b>13</b> |
| 9.1       | Chain: leaked key to full compromise . . . . .               | 13        |
| 9.2       | Chain: SSRF to credential theft . . . . .                    | 13        |
| 9.3       | Chain: public S3 bucket to sensitive data exposure . . . . . | 13        |
| <b>10</b> | <b>Security Hub and Compliance</b>                           | <b>14</b> |
| 10.1      | What Security Hub does . . . . .                             | 14        |
| 10.2      | IAM Access Analyzer . . . . .                                | 14        |
| <b>11</b> | <b>Practical Detection Recipes</b>                           | <b>15</b> |
| 11.1      | Detecting a leaked and abused access key . . . . .           | 15        |
| 11.2      | Detecting IAM privilege escalation attempts . . . . .        | 15        |
| 11.3      | Detecting public S3 exposure . . . . .                       | 15        |
| 11.4      | Detecting instance metadata abuse . . . . .                  | 15        |
| <b>12</b> | <b>Incident Response in AWS</b>                              | <b>16</b> |
| 12.1      | First steps on a suspected compromise . . . . .              | 16        |
| 12.2      | Useful containment actions . . . . .                         | 16        |

# IAM Fundamentals

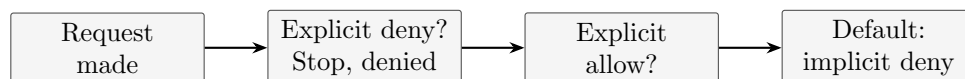
## 1.1 Why IAM is the foundation

Almost every AWS security control ultimately depends on Identity and Access Management — who can do what, to which resources — which is why IAM misconfigurations are disproportionately represented in real cloud security incidents compared to any single service’s own settings.

## 1.2 Core IAM concepts

| Concept | Meaning                                                                                          |
|---------|--------------------------------------------------------------------------------------------------|
| User    | A long-term identity, typically for a person or a service needing standing credentials           |
| Role    | A temporary identity assumed by a user, service, or another account, with its own permission set |
| Policy  | A JSON document defining what actions are allowed or denied on which resources                   |
| Group   | A collection of users sharing the same attached policies                                         |

## 1.3 How permissions are actually evaluated



**Figure 1.1.** *An explicit deny anywhere always wins; otherwise, at least one explicit allow is required, or the request is denied by default.*

### Default deny is the whole security model

Unlike some permission systems, AWS IAM denies by default — a user with no attached policies at all can do nothing. Every capability has to be explicitly granted, which is what makes over-broad policies (rather than missing ones) the far more common real-world problem.

## 1.4 Reading a policy document

### A minimal IAM policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::my-bucket/*"
    }
  ]
}
```

| Field     | Meaning                                                        |
|-----------|----------------------------------------------------------------|
| Effect    | Allow or Deny                                                  |
| Action    | The specific API call(s) this statement applies to             |
| Resource  | Which resource(s), by ARN, the statement applies to            |
| Condition | Optional additional constraints (e.g. source IP, MFA presence) |

## 1.5 The principle of least privilege

- Grant only the specific actions a role actually needs, not broad wildcard actions like `s3:*`
- Scope resources to specific ARNs rather than `"Resource": "*"` wherever possible
- Use conditions to further restrict when a permission applies (e.g. only from a specific VPC)
- Review and remove unused permissions on a recurring schedule, not just at creation

# IAM Privilege Escalation

## 2.1 Why IAM escalation matters

A low-privilege set of credentials that can modify its own (or another identity's) permissions is functionally equivalent to admin access — just requiring one extra step. Recognizing these paths is one of the highest-value skills in AWS security assessment.

## 2.2 Common escalation paths

| Permission held                                            | Escalation path                                                                                           |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>iam:CreatePolicyVersion</code>                       | Create a new version of an existing policy granting full access                                           |
| <code>iam:AttachUserPolicy</code>                          | Attach an existing admin-level policy directly to your own user                                           |
| <code>iam:CreateAccessKey</code>                           | Create new access keys for a more privileged user                                                         |
| <code>iam:PassRole</code> + assumable service              | Pass a highly privileged role to a service you control, then use that service to act with its permissions |
| <code>lambda:CreateFunction</code> + <code>PassRole</code> | Create a Lambda function with a privileged execution role, then invoke it                                 |

A minimal escalation, if `AttachUserPolicy` is held

```
aws iam attach-user-policy \
--user-name my-user \
--policy-arn arn:aws:iam::aws:policy/AdministratorAccess
```

### Enumerate your own permissions first

Before assuming an identity is genuinely low-privilege, enumerate its actual attached and inline policies directly — a role that looks narrowly scoped by name (like `read-only-role`) can still hold one of these escalation permissions if policies were attached without careful review.

## 2.3 The fix

- Restrict `iam:*` permissions to a small, dedicated set of administrative identities
- Use permission boundaries to cap what a role can grant itself even if it holds IAM-modifying permissions
- Require MFA for any identity with sensitive IAM permissions
- Regularly audit for these specific dangerous permission combinations, not just individual overly-broad policies

## S3 Security

### 3.1 Why S3 misconfigurations are so common

S3 buckets are easy to create and easy to accidentally expose — a single overly permissive bucket policy or an ACL set to public can expose an entire bucket’s contents to the internet, and this specific mistake accounts for a disproportionate share of publicly disclosed cloud data breaches.

### 3.2 Layers of S3 access control

| Layer               | Scope                                                                                      |
|---------------------|--------------------------------------------------------------------------------------------|
| IAM policies        | Control what IAM identities can do to S3, account-wide                                     |
| Bucket policies     | Attached to a specific bucket, can grant or deny access to any principal, including public |
| ACLs (legacy)       | Older, object/bucket-level access control, largely superseded by bucket policies           |
| Block Public Access | An account/bucket-level setting that overrides other settings to prevent public access     |

#### Checking a bucket’s public access block status

```
aws s3api get-public-access-block --bucket my-bucket
```

#### Block Public Access is the strongest single control

Enabling Block Public Access at the account level overrides any bucket policy or ACL that would otherwise make a bucket public — it’s the single most effective control against accidental S3 exposure, and AWS enables it by default for new buckets specifically because of how common this mistake historically was.

### 3.3 Finding exposed buckets during an assessment

#### Checking a bucket policy directly

```
aws s3api get-bucket-policy --bucket target-bucket
aws s3api get-bucket-acl --bucket target-bucket
```

#### Testing anonymous access without any credentials

```
curl https://target-bucket.s3.amazonaws.com/
aws s3 ls s3://target-bucket --no-sign-request
```



## 3.4 The fix

- Enable Block Public Access at the account level unless a bucket has a genuine, reviewed need for public content
- Use bucket policies with explicit, narrowly scoped principals rather than relying on ACLs
- Enable S3 server access logging and/or CloudTrail data events for sensitive buckets
- Encrypt bucket contents at rest and enforce encryption in transit via bucket policy conditions

## EC2 and Compute Security

### 4.1 The instance metadata service

Every EC2 instance can query a local, non-routable endpoint for metadata about itself — including, critically, temporary credentials for any IAM role attached to the instance.

#### Querying instance metadata, IMDSv1 style

```
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/
```

#### Why this is such a common SSRF target

If an application running on the instance has an SSRF vulnerability (see the Web Application Security Playbook), an attacker can often reach this metadata endpoint through the vulnerable application itself, harvesting the instance's IAM role credentials without ever needing direct access to the instance — one of the most consequential SSRF outcomes possible.

### 4.2 IMDSv1 vs. IMDSv2

| Version | Behavior                                                                                                     |
|---------|--------------------------------------------------------------------------------------------------------------|
| IMDSv1  | Simple GET request, no token required — vulnerable to basic SSRF                                             |
| IMDSv2  | Requires a session token obtained via a PUT request first, which most simple SSRF techniques can't replicate |

#### Enforcing IMDSv2 only

```
aws ec2 modify-instance-metadata-options \
  --instance-id i-0123456789abcdef0 \
  --http-tokens required
```

### 4.3 Security groups

Security groups act as a stateful, instance-level firewall — the most common misconfiguration is an inbound rule allowing 0.0.0.0/0 on a sensitive port (SSH, RDP, a database port), exposing the instance directly to the entire internet.

#### Auditing for overly permissive security groups

```
aws ec2 describe-security-groups \
  --filters Name=ip-permission.cidr,Values=0.0.0.0/0
```

## 4.4 The fix

- Enforce IMDSv2 fleet-wide via account-level defaults for new instances
- Restrict security group inbound rules to specific, known source ranges rather than the open internet
- Attach IAM roles scoped to exactly what the instance's workload requires, nothing broader

## VPC and Network Security

### 5.1 Core VPC concepts

| Concept          | Role                                                                                               |
|------------------|----------------------------------------------------------------------------------------------------|
| VPC              | An isolated virtual network within an AWS account/region                                           |
| Subnet           | A segment of a VPC's address range, tied to a single availability zone                             |
| Route table      | Determines where traffic from a subnet is directed                                                 |
| Internet Gateway | Provides a route to the public internet for a VPC                                                  |
| NAT Gateway      | Lets private-subnet resources reach the internet outbound without being directly reachable inbound |

### 5.2 Public vs. private subnets

A subnet is "public" only because its route table sends 0.0.0.0/0 traffic to an Internet Gateway — it's a routing decision, not an inherent property, which is exactly why auditing route tables directly (not just assuming based on a subnet's name) matters during a security review.

### 5.3 VPC Flow Logs

#### Enabling flow logs for a VPC

```
aws ec2 create-flow-logs \
  --resource-type VPC --resource-ids vpc-0123456789abcdef0 \
  --traffic-type ALL \
  --log-destination-type cloud-watch-logs \
  --log-group-name vpc-flow-logs
```

Flow logs record metadata about traffic crossing network interfaces — source/destination, port, protocol, and whether it was accepted or rejected — the VPC-level equivalent of NetFlow data, and often the first place to look when investigating unexpected network activity.

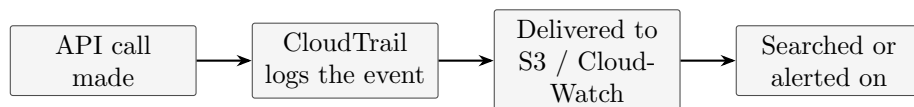
#### Flow logs don't capture payload

Flow logs record that a connection happened, not what was actually transferred — for payload-level visibility, this needs to be paired with traffic mirroring or host-based tooling, not relied on alone.

# CloudTrail

## 6.1 What CloudTrail records

CloudTrail logs every API call made against an AWS account — who made it, when, from where, and what the request/response looked like — providing the audit trail that essentially every AWS investigation starts from.



**Figure 6.1.** Every single AWS API call, from any source (console, CLI, SDK), passes through this same pipeline.

## 6.2 Management vs. data events

| Event type        | Covers                                                                       |
|-------------------|------------------------------------------------------------------------------|
| Management events | Control-plane operations: creating/modifying/deleting resources, IAM changes |
| Data events       | Data-plane operations: S3 object-level access, Lambda invocations            |

### Data events cost extra and aren't on by default

Management events are logged by default at no additional charge; data events (like individual S3 `GetObject` calls) require explicit configuration and carry additional cost — meaning object-level S3 access visibility is an opt-in decision, not something to assume is already being captured.

## 6.3 Key fields for investigation

### A CloudTrail event, key fields

```

eventName: ConsoleLogin
eventTime: 2026-01-15T03:42:11Z
sourceIPAddress: 203.0.113.45
userIdentity.arn: arn:aws:iam::123456789012:user/jdoe
userAgent: aws-cli/2.x
  
```

## 6.4 Querying CloudTrail

Athena query example, once logs are queryable as a table

```
SELECT eventtime, eventname, sourceipaddress, useridentity.arn
FROM cloudtrail_logs
WHERE eventname = 'ConsoleLogin'
AND eventtime > '2026-01-01'
ORDER BY eventtime DESC
```

# GuardDuty and Threat Detection

## 7.1 What GuardDuty does

GuardDuty is AWS's native, managed threat detection service — it continuously analyzes CloudTrail, VPC Flow Logs, and DNS logs, applying threat intelligence and anomaly detection without requiring any agent installation or log pipeline setup of your own.

## 7.2 Finding categories

| Category              | Example                                                              |
|-----------------------|----------------------------------------------------------------------|
| Reconnaissance        | Port scanning, unusual API call patterns consistent with enumeration |
| Instance compromise   | An EC2 instance communicating with known malicious infrastructure    |
| Credential compromise | API calls from a Tor exit node, or geographically impossible travel  |
| S3 threats            | Anomalous data access patterns, suspicious API call volume           |

### Enabling GuardDuty is close to a free win

GuardDuty requires no agents and analyzes data sources AWS already generates — for the detection coverage it provides, it's one of the highest-value, lowest-effort security controls available in any AWS account, and there's little reason not to enable it account-wide.

## 7.3 Responding to a GuardDuty finding

- Pull the full finding detail, including the specific resource and API calls involved
- Cross-reference CloudTrail directly for the surrounding activity from the same identity/IP
- Determine scope: one resource, one identity, or broader account compromise
- For credential compromise findings, rotate the affected credentials immediately as a first step

# Secrets Management

## 8.1 Where secrets go wrong

- Hardcoded credentials committed directly to a source repository
- Access keys embedded in EC2 user-data or container environment variables
- Long-lived IAM user access keys instead of temporary role-based credentials
- Secrets logged accidentally through verbose application logging

## 8.2 AWS Secrets Manager and Parameter Store

| Service                         | Notes                                                                         |
|---------------------------------|-------------------------------------------------------------------------------|
| Secrets Manager                 | Built-in rotation support, tighter integration with RDS and other services    |
| Systems Manager Parameter Store | Simpler, cheaper for static configuration values, supports encryption via KMS |

Retrieving a secret at runtime instead of hardcoding it

```
aws secretsmanager get-secret-value --secret-id prod/db/password
```

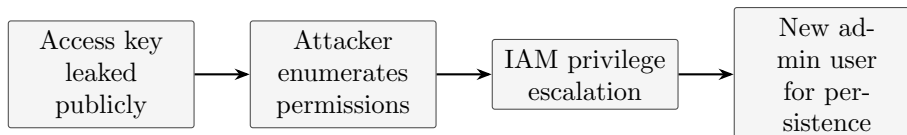
Prefer roles over long-lived keys entirely

The strongest secrets management practice for AWS-to-AWS communication is often having no long-lived secret at all — IAM roles provide automatically rotated, short-lived temporary credentials, removing the entire class of "leaked static credential" risk for workloads running on AWS compute.



## Common Cloud Attack Chains

### 9.1 Chain: leaked key to full compromise



**Figure 9.1.** *A single leaked key, often from a public code repository, is a common starting point for this exact chain.*

### 9.2 Chain: SSRF to credential theft

- A web application with an SSRF vulnerability is identified
- The vulnerability is used to reach the EC2 instance metadata endpoint
- The attached IAM role’s temporary credentials are harvested from the response
- Those credentials are used directly against the AWS API, external to the compromised instance

### 9.3 Chain: public S3 bucket to sensitive data exposure

- A bucket is discovered via subdomain enumeration or a leaked reference
- Bucket policy or ACL review (or a direct anonymous request) confirms public read access
- Bucket contents are listed and downloaded directly, no credentials required

#### The common thread

Nearly every chain in this chapter starts from a single, individually unremarkable weakness — one leaked key, one SSRF bug, one public bucket — and becomes serious only because it connects to a broader identity or network context. This is exactly why IAM hygiene and network segmentation matter even for resources that seem low-value on their own.

## Security Hub and Compliance

### 10.1 What Security Hub does

Security Hub aggregates findings from GuardDuty, Inspector, IAM Access Analyzer, and other AWS security services into one dashboard, and can continuously check an account's configuration against standards like the CIS AWS Foundations Benchmark.

#### Enabling a compliance standard

```
aws securityhub batch-enable-standards \
--standards-subscription-requests \
StandardsArn=arn:aws:securityhub:::ruleset/cis-aws-foundations-benchmark/v/1.2.0
```

### 10.2 IAM Access Analyzer

Access Analyzer specifically identifies resources (S3 buckets, IAM roles, KMS keys, and more) that are accessible from outside the account — surfacing exactly the kind of unintended external exposure that manual policy review is prone to missing.

#### Compliance tooling finds gaps, not intent

A clean CIS benchmark score doesn't mean an account is free of security issues — these checks catch known misconfiguration patterns, not novel logic flaws or business-context-specific risks, so compliance tooling is a floor, not a ceiling, for a real security review.

## Practical Detection Recipes

### 11.1 Detecting a leaked and abused access key

#### CloudTrail query pattern to watch for

```
-- Sudden API activity from a new, unfamiliar source IP
-- for an access key with a long history of only ever
-- being used from one known location
```

### 11.2 Detecting IAM privilege escalation attempts

#### CloudTrail events worth alerting on

```
eventName = AttachUserPolicy
eventName = PutUserPolicy
eventName = CreatePolicyVersion
eventName = CreateAccessKey (for a user other than the caller)
```

### 11.3 Detecting public S3 exposure

#### Automated checks worth scheduling

```
aws s3api get-public-access-block --bucket <each-bucket>
-- flag any bucket where BlockPublicAcls or
-- RestrictPublicBuckets is false
```

### 11.4 Detecting instance metadata abuse

- GuardDuty's `UnauthorizedAccess:EC2/MetadataDNSRebind` and related findings cover this directly
- Cross-reference any IAM role credential use from an unexpected source IP against known instance IPs

# Incident Response in AWS

## 12.1 First steps on a suspected compromise

- Identify and rotate or disable the specific compromised credential immediately
- Preserve evidence: CloudTrail logs, GuardDuty findings, and affected resource state, before making further changes
- Isolate affected resources (e.g. via security group changes) rather than terminating them immediately, to preserve forensic value
- Determine the full scope: what else could this identity or resource have touched

## 12.2 Useful containment actions

| Action                                        | Effect                                                                          |
|-----------------------------------------------|---------------------------------------------------------------------------------|
| Revoke active sessions for a compromised role | Invalidates temporary credentials already issued                                |
| Attach a deny-all IAM policy                  | Immediately blocks further action by a compromised identity without deleting it |
| Isolate an EC2 instance's security group      | Cuts network access while preserving the instance for forensics                 |
| Enable S3 Object Lock on relevant buckets     | Prevents further tampering with data during investigation                       |

### Rotating credentials isn't the same as understanding scope

Rotating a leaked key stops further abuse with that specific credential, but doesn't answer what was already done with it — a full CloudTrail review of that identity's activity during the compromise window is still necessary before considering an incident actually closed.